
LECTURE NOTES

Introduction to Machine Learning

AI512 — Introduction to Machine Learning

Simon Holm
2026-09-04



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1	Basic Concepts	4
1.1	Formal definitions	4
1.2	Empirical Risk Minimization (ERM)	4
1.3	Polynomial curve fitting	5
2	Linear Predictors	5
2.1	Least Squares Regression	5
2.2	Metric Spaces	6
2.3	Regularized Least Squares	7
2.4	Ridge regression	8
2.5	Z-score normalization	8
2.6	Lasso regression	9
3	Classification	12
3.1	Binary Classification	12
3.2	Logistic Regression	13
3.3	Multi-Class Classification	14
3.4	Performance Metrics for Classification	15
3.5	K-Fold Cross Validation	15
3.6	K-Nearest Neighbors (kNN) Classifier	15
3.7	Receiver Operating Characteristics (ROC)	16
4	Probability Theory	16
4.1	Basic Concepts	16
4.2	Random Variables	19
4.3	Expectance and Variance	20
4.4	Continuous Random Variables	21
4.5	Common Distributions	24
4.6	Estimators And Bias	26
4.7	Inequalities	26
5	Statistical Learning Theory	28
5.1	Generalization bounds	28
5.2	PAC Learnability	29
5.3	Bias-Complexity Dilemma (Trade-off)	31
5.4	Vapnik - Chervonenkis (VC) Dimension	33
5.5	Nonuniform Learnability	35
6	Bayesian Learning	36
6.1	Maximum Likelihood Estimation (MLE)	36
6.2	Bayesian Learning	38
6.3	Maximum A-Posteriori Estimation	40
6.4	Monte Carlo Integration	41
6.5	Generative Models	42
6.6	Naive Bayes Classifier	42
7	Kernel Methods	43
7.1	Transforming the input space	43

7.2	Kernel Trick	43
7.3	Kernel regression	44
7.4	Support Vector Machines (SVM)	44
8	Ensemble Methods	44
8.1	Bootstrap Aggregation (Bagging)	44
8.2	Boosting	44
8.3	Federated Learning	44

1 Basic Concepts

1.1 Formal definitions

1.1.1 Formal definitions

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured in P , improves with experience E ”. [Mitchell, 1997]

So

1.1.1.1 Experience E

Experience E comes from the dataset itself $S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$.

The goal is to find a hypothesis that accurately represents $S := \{(x_i, y_i) \stackrel{i.i.d}{\sim} D\}$

1.1.1.2 Task T

The task is to predict $f(x_1) = y_i$ with $h(x_i) \approx y_i$

$$f(x) \approx h(x), \forall x \in X$$

Task can either be a **Classification** or a **Regression** dependent on whether Y is a continuous set (like $\mathbb{R}$)

1.1.1.3 Performance measure P

Performance is measure via a loss(risk) function $L : Y \times Y \rightarrow \mathbb{R}^+$ Where $\ell(y, y')$

where y is the true value of y and $\hat{y}$ is the prediction of y

Loss (risk) - measures how bad a single prediction $\hat{y}$ is compared to the true label y .

We define two common choices

- $\ell(y, \hat{y}) := \mathbb{I}(y \neq \hat{y})$, zero-one loss for classification
- $\ell(y, \hat{y}) := (y - \hat{y})^2$, squared error for regression

1.2 Empirical Risk Minimization (ERM)

Take the zero-one loss $\ell(y, y') = \mathbb{I}(y \neq y')$ as an example

Then we want to minimize $L_{D,f}(h) := \mathbb{P}_{x \sim D}[f(x) \neq h(x)]$

$$h_* = \arg \min_h L_{D,f}(h)$$

Since $f(x)$ is unknown we can find the average using actual true results

$$h_S = \arg \min_h L_S(h) = \arg \min_h (\text{abs}(\{i \in [m], h(x) - f(x)\}))/m$$

This is not well defined and has weaknesses regarding the hypothesis. We resolve to curve fitting.

1.3 Polynomial curve fitting

With dataset $S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ it is wise to split data into the following:

- Training set, where the learning algorithm will find a h that minimizes $L(h)$
- Test set, check how well the algorithm is doing (are we closer to approximating $f(x)$?)

This is ofc assuming i.i.d.

We can use polynomial fitting instead

$y(x) = \sum_{m=0}^M w_m x^m$ Then by **ridge regression** $w = (X^T X + \lambda I)^{-1} X^T Y$ We can predict y by $x^T w$ (example of this)

For each order (m) we can calculate the risk with the **Root Mean Squared Error (RMSE)**

$$L_S := \sqrt{\frac{1}{m} \sum_{i \in [m]} (h(x_i) - y_i)^2}$$

(example)

This should by theory then go down (risk should decrease as the algorithm learns)

Notice that at $M = 7$ the model overfits and acts poorly

This is because of very small or large weights

- at a small m , model will likely underfit
- at a high m , model will likely overfit

To avoid this we use a **regularizer** $h_S := \arg \min_h L_S(h) + \lambda w_m^2$ This mitigates over/under-fitting as polynomial degree increases

2 Linear Predictors

2.1 Least Squares Regression

Assume a dataset Data obviously has a linear correlation. So we can denote the hypothesis space more specifically

$$\mathcal{H} := \{h : h(x) = w^T x, w \in \mathbb{R}^k\}$$

Where $w = (w_0, w_1)$

Assume dataset $S = \{(x_i, y_i) : i \in [m]\}$, then squared-error loss for h is:

$$L_S(w) := \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2$$

Then $w_S := \arg \min_w L_S(w)$ to find a good vector for the hypothesis w

$$\begin{aligned}
\nabla_w L_S(w) &= \frac{2}{m} \sum_{i=1}^m x_i (x_i^\top w - y_i) \\
&= 0 \\
\Rightarrow \sum_{i=1}^m x_i x_i^\top w_S &= \sum_{i=1}^m x_i y_i \\
\Rightarrow w_S &= \left(\sum_{i=1}^m x_i x_i^\top \right)^{-1} \sum_{i=1}^m x_i y_i
\end{aligned}$$

This is often referred to as least squares regression, and w as the least squares solution.

We can denote $z_i := (x_i, 1)$, where $Z = \begin{pmatrix} z_1 \\ \vdots \\ z_n \end{pmatrix}$ to show that:

$$w_S := (Z^\top Z)^{-1} Z^\top y.$$

(this is an example of the learned hypothesis h with w)

2.2 Metric Spaces

We would like the vector spaces (such as feature spaces, parameters spaces etc.) to have some plausible properties such as.

- **strict:** $\forall a, a' \in X, a \neq a' \Rightarrow \text{dist}(a, a') > 0$,
- **reflexive:** $\forall a \in X, \Rightarrow \text{dist}(a, a) = 0$,
- **symmetric:** $\forall a, a' \in X, \Rightarrow \text{dist}(a, a') = \text{dist}(a', a)$.
- **triangle inequality:** $\forall a, b, c \in X, \text{dist}(a, c) \leq \text{dist}(a, b) + \text{dist}(b, c)$.

Where $\text{dist}(a, b)$ is a function that measures distance between two points.

These requirements are nice to have since they make the distance function $\text{dist}(a, b)$ behave like a **real notion of distance**.

Using this we can define the **norm** of a vector aka. The length For $p > 0$ the L_p norm is defined as follows:

$$\|u\|_p = \left(\sum_{j=1}^k |u_j|^p \right)^{1/p}$$

If $p = 2$, we get the well-known **Euclidean norm**.

If $p = 1$, we get the **Manhattan norm**.

As $p \rightarrow \infty$, we get the **Maximum norm** (L_∞), i.e. $\|u\|_\infty := \max\{|u_1|, \dots, |u_d|\}$.

We can use this norm to derive many nice metric spaces

$$\text{dist}(a, b) = \|a - b\|_p.$$

(this example shows the behavior of the distances w.r.t p)

2.3 Regularized Least Squares

As we saw in 3. Polynomial curve fitting, $L_S(h_S) = 0$ by memorizing training data can be achieved but will result in overfitting

Assume a hypothesis space H of finite size $|H|$ and a loss function $L_s(h)$ which fits the data as well as possible and constraining model complexity.

$$h_S := \arg \min_{h \in H} L_S(h) + \lambda |H|$$

With λ as a **regularization coefficient** and $|H|$ the **regularizer** this is called **Structured Risk Minimization**, one of the biggest achievements of machine learning research in the pre-deep-learning era.

Let us apply this to the least squares problem.

$$h_S := \arg \min_w \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2$$

As we know, when m become very large, the weights will also become large, and thus result in overfitting. To counter this we force the model to keep weights small with the constraint

$$h_S := \arg \min_w \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2, \quad ||w||_p \leq \eta$$

This constrained optimization problem can be expressed equivalently as:

$$h_S := \arg \min_w \max_{\lambda} \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2 + \lambda (||w||_p^p - \eta)$$

where $\lambda \geq 0$.

By choosing a large λ and dropping the inner max problem, we can find a reasonable approximation referred to as **regularized least squares**:

$$h_S := \arg \min_w \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2 + \lambda ||w||_p^p$$

We can solve for w with $\nabla L_S(h_S) = 0$ to find a good w for the hypothesis

2.4 Ridge regression

Take a special case of regularized least squares, where $p = 2$ (example of solving for an opt. w)

$$h_S := \arg \min_w \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2 + \lambda \|w\|_2^2$$

We can rewrite the loss of this optimization in vector form as.

$$L_S(w) := \frac{1}{m} (Zw - y)^2 + \lambda w^\top w$$

Let us find the optimal weights that minimize the loss by setting its gradient to zero once again: (skipped because I can't be bothered smh)

$$\begin{aligned} L_S(w) &= \frac{1}{m} w^\top Z^\top Z w - \frac{1}{m} 2w^\top Z^\top y + \lambda w^\top w \\ \Rightarrow w_S &= \left(\frac{1}{m} Z^\top Z + \lambda I \right)^{-1} Z^\top y. \end{aligned}$$

This again is known as **ridge regression**.

Note: finding a lambda which serves an optimal penalty, is normally found with **k-fold Cross Validation**

2.5 Z-score normalization

A model outputs the following:

Means: [-0.00031786 0.00045392 -0.00103062 -0.00226024 -0.00069973 -0.00092399
-0.00058225 0.00039851 0.00066739 -0.00087822] Variances: [0.00222035 0.00226498
0.00217988 0.00219522 0.00233499 0.00234279 0.00229155 0.00229439 0.0023026 0.00222032]

Clearly these means vary a lot in scales (-0.0023 is almost 10x larger than -0.0003)

To avoid unintentionally prioritizing the features with larger numbers we can normalize (using **Z-score normalization**)

Assume that some data is normal distributed with some mean μ and variance σ^2 . That is, each dimension x_d of a D -dimensional observation x comes from a sampling process as below:

$$\epsilon \sim N(0, 1), \quad x_d = \mu_d + \sigma_d \epsilon$$

We can then

$$x' = \frac{x_d - \mu_d}{\sigma_d} = \epsilon$$

And thus we can center and scale based on the sample mean μ and standard deviation σ .

2.6 Lasso regression

Much like ridge regression where

$$h_S := \arg \min_w \frac{1}{m} \sum_{i \in [m]} (w^\top x_i - y_i)^2 + \lambda \|w\|_2^2$$

Where the penalty term is $\lambda \|w\|_2^2$

In Lasso regression, the penalty term is simply $\lambda |w|$

This way Lasso “selects” features by letting only the most predictive ones survive while others are pushed exactly to zero. This is unlike ridge regression, where terms can only approach 0.

So with Lasso regression

- If a feature is **strongly correlated with the output**, it tends to survive (keep a nonzero coefficient).
- If a feature is **weakly correlated** or **highly redundant compared with other features**, Lasso often sets it to zero.

This however results in a problem since,

$$f'(x) = \begin{cases} 1, & \text{if } x > 0 \\ [2mm] - 1, & \text{if } x < 0 \\ [1mm] \text{undefined,} & \text{if } x = 0 \end{cases}$$

Because of this we cannot solve for w using $\nabla L_S(h_S) = 0$

However, we can use gradient descent in the direction $-\nabla L_S(h_S)$ (the direction opposite of the greatest uphill) to approach an optimal w where $\nabla L_S(h_S)$ is as small as possible.

$$w_{t+1} := w_t - \alpha \nabla_w L_S(w) \big|_{w:=w_t}$$

This approach is called **gradient descent**. It is in use with nearly all modern machine learning approaches. The coefficient $\alpha > 0$ is called a **learning rate**.

Gradient descent requires repetitive evaluation of the gradient of the loss with respect to the parameters at every iteration: $L_S(w) \big|_{w:=w_t}$. Hence, its implementation on the computer requires an analytical calculation of this gradient. This may be time consuming for complex loss functions. Deep learning libraries such as PyTorch and TensorFlow allow

us to automate this process. See an example PyTorch implementation of Lasso regression below.

2.6.1 From 02_Linear_Predictors.ipynb

```
1  import torch
   as th
2  import torch.nn as nn
3  import torch.nn.functional as F
4  import torch.optim
5
6  class LassoRegression(nn.Module):
7      def __init__(self, n_dims, lambda_coef=1):
8          super(LassoRegression, self).__init__()
9          self.lambda_coef = lambda_coef
10         self.emp_risk = nn.MSELoss()
11         self.weight = nn.Parameter(th.randn((n_dims,1)))
12         self.bias = nn.Parameter(th.randn((1)))
13
14     def predict(self, input):
15         return input @ self.weight + self.bias
16
17     def learn(self, inputs, labels, num_steps=1):
18         # The in-built Stochastic Gradient Descent optimizer
19         # The argument "lr" sets the learning rate
20         optimizer = torch.optim.SGD(self.parameters(), lr=0.01)
21
22         for ii in range(num_steps):
23             # Predict with the current weight values
24             # This step is called a "forward pass"
25
26             predictions = self.predict(inputs)
27             loss = ((predictions - labels)**2).mean() \
28                 + self.weight.abs().sum()*self.lambda_coef
29
30             # Clear the gradient values remaining from
31             # the previous iteration
32             optimizer.zero_grad()
33
34             # Compute the new gradient values
```

```

35     # This step is called the "backward pass"
36     loss.backward()
37
38     # Take the gradient descent step
39     optimizer.step()
40
41 # Convert data into the Torch format
42 X_train = torch.tensor(X_train).float()
43 X_test = torch.tensor(X_test).float()
44 y_train = torch.tensor(y_train).float().reshape(-1,1)
45 y_test = torch.tensor(y_test).float().reshape(-1,1)
46
47 # z-score normalization
48 m = th.mean(X_train,axis=0)
49 std = th.std(X_train,axis=0)
50 X_train = (X_train-m)/std
51 X_test = (X_test-m)/std
52 # Train our model
53 model_lasso = LassoRegression(n_dims=X_train.shape[1], lambda_coef=1)
54
55 # Number of gradient descent iterations
56 num_iterations = 250
57
58 # Collect the train and test errors here.
59 train_errors = np.zeros(num_iterations)
60 test_errors = np.zeros(num_iterations)
61
62 for ii in range(num_iterations):
63     model_lasso.learn(X_train, y_train)
64     predictions = model_lasso.predict(X_train)
65     train_error = ((predictions - y_train)**2).mean().sqrt()
66     train_errors[ii] = train_error.detach().numpy()
67
68 # Test our model
69 predictions = model_lasso.predict(X_test)
70 test_error = ((predictions - y_test)**2).mean().sqrt()
71 test_errors[ii] = test_error.detach().numpy()
72

```

```

73 # Plot the learning curve
74 plt.plot(np.arange(num_iterations),train_errors,'b-', label="Train
    RMSE")
75 plt.plot(np.arange(num_iterations),test_errors,'r-', label="Test RMSE")
76 plt.xlabel("Iteration")
77 plt.ylabel("RMSE")
78 plt.legend(loc="upper right")
79 plt.show()

```

(plotted graph from lecture)

3 Classification

3.1 Binary Classification

Assume that we have a binary classification problem where

$$S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}, \text{quad} \text{ and } \text{quady} \text{ in } \{0, 1\}$$

Like before we assume the hypothesis to be linear:

$$\mathcal{H} := \{h : h(x) = w^\top x, w \in \mathbb{R}^k\}$$

But in this case the output is now discrete and not continuous.

Let's define one possible way to interpret the label y .

- Sign of h_i
 - If $h_i > 0 \rightarrow$ class 1
 - If $h_i < 0 \rightarrow$ class 0
- magnitude of $|h_i|$
 - Larger $|h_i| \rightarrow$ higher **confidence**
 - Smaller $|h_i| \rightarrow$ lower **confidence**

Note: this is just **one** possible way of interpreting a discrete output.

When interpreted this way, the hypothesis h_i is called the **discriminant function**. Such a model is not easy to train since we normally desire a differentiable loss function, which we can't with this "sign" function. By considering this a probability problem would like to achieve:

$$p_i = w^T x_i \quad \text{WRONG!}$$

The problem with this is that the range of $w^T x_i \in (-\infty, \infty)$ and $p_i \in [0, 1]$

By taking the log we can widen this so.

$$\log(p_i) \propto w^T x_i$$

This works well for any $w^T x_i < 0$, but is undefined for $w^T x_i > 0$ since $\log(a) \in (-\infty, 0)$. To fix this, we account for the probability of the other class. We actually want to express

$$w^T x_i = \log\left(\frac{\mathbb{P}(y_i=1|x_i)}{\mathbb{P}(y_i=0|x_i)}\right) = \log\left(\frac{p_i}{1-p_i}\right)$$

This is called logistic function, and we can solve for p_i to find probabilities for both classes.

$$\log\left(\frac{p_i}{1-p_i}\right) = w^T x_i \Rightarrow p_i = \frac{1}{1+e^{-w^T x_i}}$$

3.2 Logistic Regression

We have learned how binary classification can be done using probability. From that we get that:

$$\log\left(\frac{p_i}{1-p_i}\right) = w^T x_i \Rightarrow p_i = \frac{1}{1+e^{-w^T x_i}}$$

The function $\sigma(u) = \frac{1}{1+e^{-u}}$ is known as the sigmoid function, and the inverse of the sigmoid function is

$$\sigma^{-1}(p) = \frac{p}{1-p} \quad \text{for some } p, \text{ given that } u = w^T x_i$$

This is called logistic regression.

Now to determine the probability of the whole dataset. We use the following notation

$$\mathbb{P}(y_i | x_i, w) = p_i^{y_i} (1 - p_i)^{1-y_i}$$

This way:

- $\mathbb{P}(y_i = 0 | x_i, w) = 1 - p_i$
- $\mathbb{P}(y_i = 1 | x_i, w) = p_i$

(This is an example of how logistic regression uses probability to determine whether mice are obese or not)

And so to determine the whole dataset we can

$$\prod_{i=1}^m p_i^{y_i} (1 - p_i)^{1-y_i}$$

Note that products are messy to differentiate, which is nice, for optimization, therefore we can express this as a sum instead:

$$\ell(w) = \log\left(\prod_{i=1}^m p_i^{y_i} (1 - p_i)^{1-y_i}\right) = \sum_{i=1}^m [y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)]$$

Now to express this as a loss function we take the negative log-likelihood, and substitute $p_i = \sigma(w^T x_i)$ (the sigmoid)

$$\mathcal{L}_{01}(w) = - \sum_{i=1}^m \log \sigma(w^\top x_i)^{y_i} (1 - \sigma(w^\top x_i))^{1-y_i}$$

Now we can differentiate the loss function w.r.t w to find an optimal w . That is stupid, so I won't do it here.

3.3 Multi-Class Classification

Assume that we now have more than 2 classes (so not binary anymore). Assume that we have C classes, so that

$$S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}, \text{quad"and"quad} y_i \in \{0, 1, C-1\}$$

We will need to model the class probabilities of C different classes: $p_1, p_2, \dots, p_C$:

In binary classification, we discriminated classes with a **sign function**. In multi-class classification we ought to assign a **score for each class**:

$$s_c = w_c^\top x_i, \quad c = 1, 2, \dots, C$$

Where $s_c \in (-\infty, \infty)$

We want to map these scores to probability. When mapping to probability, we want to make sure that the following constraints hold:

- $0 \leq p_i \leq 1$, each probability is constraint as a number between 1 and 1
- $\sum_{c=1}^C p_c = 1$, the sum of all p_c is always 1.

Softmax deals with exactly this, by generalizing the binary classification to $C > 2$

$$\sigma(x)_c = \frac{e^{x_i}}{\sum_{j=1}^k e^{x_j}}$$

The related loss function is then:

$$\mathcal{L}_{CE}(W) = - \sum_{i=1}^m \log \sigma(w_{y_i}^\top x_i) = \sum_{i=1}^m \left\{ -w_{y_i}^\top x_i + \log \left(\sum_{c=1}^C e^{w_c^\top x_i} \right) \right\}$$

where $W = [w_1 \dots w_C]$ is a matrix of the weight vectors for each class. This loss function is called the **cross entropy loss**. We will revisit this loss and understand better why it is given this particular name.

Let's try and make an as general algorithm as possible using what we have learned from regularization

$$L(W) := L_{CE}(W) + \lambda \sum_{c=1}^C \|w_c\|_p$$

Like before we can use gradient descent to minimize loss (example from lecture)

3.4 Performance Metrics for Classification

The goal of classification is to recognize a pattern. We introduce confusion matrices. For confusion matrices:

- TP true and correct class
- FP: false and correct class
- TN true and incorrect class
- FN: false and incorrect class

We can compute many performance metrics from the confusion matrix:

- Accuracy $:= \frac{\text{Sum of Diagonal}}{\text{Sum of all samples}}$
- Precision (per class) $:= \frac{TP}{TP+FP}$
- Recall (per class) $:= \frac{TP}{TP+FN}$
- F1 Score (per class) $:= F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
- False Positive Rate (per class) $:= \frac{FP}{FP+TN}$
- Specificity $:= \frac{TN}{TN+FP}$

3.5 K-Fold Cross Validation

Split up data in k different ways, to optimize

3.6 K-Nearest Neighbors (kNN) Classifier

kNN is based around memorizing the entire dataset and using distance to neighboring data points to classify new data.

This way a new point is classified based on the classes of its **k nearest neighbors** in the training set.

There are multiple ways to do kNN, here I will show 2 alike, yet different ways.

3.6.0.1 Standard kNN

$$\hat{y} = \arg \max_{c \in [C]} \sum_{i=1}^k \mathbb{1}(y_i = c)$$

The standard kNN selects a class to a new data point, only based on the majority vote of the k-nearest neighbors.

3.6.0.2 Weighted kNN

$$\hat{y} = \arg \max_{c \in [C]} \sum_{i=1}^k \mathbb{1}(y_i = c) \frac{1}{d(x, x_i)}$$

The weighted kNN does the same, however it normalizes each vote, using the ratio of the distance. This way, points with a small distance to the new point x contribute more to the vote, than points with greater distance to the new point.

3.6.0.3 Voronoi cells

A **voronoi cell** is the area around a datapoint to which it is the closest point. Example, any new data point within the blue area, will be classified to the blue

When you do this for all data points, you can create a Voronoi map

3.7 Receiver Operating Characteristics (ROC)

ROC curve is a tool to **evaluate the performance of a binary classifier**. Take this example, to find out where the threshold should be (for the model to be optimal), we can use ROC

To understand ROC, we need **two rates**:

1) True Positive Rate (TPR)/ sensitivity / Recall

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

2) False Positive Rate (FPR)

$$\text{FPR} = \frac{\text{FP}}{\text{TN} + \text{FN}}$$

W

The ROC plots these as follows for each: The more to better thresholds are the ones most at the top left. Now we can choose one, depending on what the model is trying to achieve.

Also, we can use the **AUC** (area under curve) as measurement for how good the classifier is. The higher the AUC, the better the classifier. A perfect classifier has an AUC of 1. A classifier that performs no better than random guessing has an AUC of 0.5.

4 Probability Theory

4.1 Basic Concepts

Intuitive meaning of probability: how often an event happens if you repeat a random experiment many times.

$$P(A) = \lim_{n \rightarrow \infty} \frac{n_A}{n}$$

4.1.0.1 Sample space

The **sample space** is all the possible outcomes.

- Example: Rolling one die $\rightarrow \Omega = 1, 2, 3, 4, 5, 6$

4.1.0.2 Event A

An **event** is just a subset of the sample space.

- Example: $A = \text{"rolling an even number"} = 2, 4, 6$

Events are what you actually care about measuring probabilities for.

We say powerset of Ω as 2^Ω , then $\sigma(\Omega) \subseteq 2^\Omega$ is an eventspace (collection of subsets of Ω). An eventspace must be closed under countable unions, countable intersections, and complements. That means:

- 1) $\bigcup_{i=1}^{\infty} A_i \in \sigma(\Omega)$
- 2) $\bigcap_{i=1}^{\infty} A_i \in \sigma(\Omega)$
- 3) $A^c \in \sigma(\Omega)$

For many discrete sample spaces such as the outcome of the roll of a pair of dice, it is possible to assume that $2^\Omega = \sigma(\Omega)$. However, for continuous sample spaces we need to define a σ -algebra that is smaller than 2^Ω . This is because an uncountable set with size 2^Ω is very unstable and breaks the rules of probability.

Example in a dart game, if we care about the precise hit of a dart arrow, this can become very complex. In dart we resort to only care about the actual points the dart arrow hits. So we make the space smaller.

A **probability measure** is a function $P : \sigma(\Omega) \rightarrow [0, 1]$ such that:

- 1) $P(\Omega) = 1$
- 2) $P(A) \geq 0$ for all $A \in \sigma(\Omega)$
- 3) If $A_1, A_2, \dots$ are disjoint events (i.e. $A_i \cap A_j = \emptyset$ for all $i \neq j$), then $P(\bigcup_{i=1}^{\infty} A_i) = \sum_{i=1}^{\infty} P(A_i)$

We actually want to restrict the events to a **well-behaved σ -algebra**

4.1.0.3 Probability Space:

A tuple defined as $(\Omega, \sigma(\Omega), P)$.

4.1.0.4 Inclusion-Exclusion Principle:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

This takes into account the fact that $P(A \cap B)$ is counted twice in $P(A) + P(B)$. A direct consequence of this is that $P(A \cup B) \leq P(A) + P(B)$ which is called the **union bound**.

4.1.0.5 Conditional Probability:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

The intuitive meaning of this is the probability of event A given that event B has occurred. That is, the frequency of event A in the subset of trials where event B has occurred. In mathematical terms

$$P(A | B) = \lim_{n \rightarrow \infty} \frac{n_{A \cap B}}{n_B}$$

The definition of conditional probability can be rewritten as $P(A \cap B) = P(A | B)P(B)$. This is called the **product rule**.

4.1.0.6 Independence:

If the probability of event A is not affected by the occurrence of event B , these two events are said to be independent.

In terms of conditional probabilities we can describe this situation as $P(A|B) = P(A)$. Applied to the definition of conditional probability, this means that A and B are independent if and only if $P(A \cap B) = P(A)P(B)$ independent

$P(A \cap B) \neq P(A)P(B)$ dependent

4.1.0.7 Law of Total Probability:

If $B_1, B_2, \dots, B_n$ is a partition of Ω , i.e. $B_i \cap B_j = \emptyset$ for all $i \neq j$ and $\bigcup_{i=1}^n B_i = \Omega$, then

$$P(A) = \sum_{i=1}^n P(A | B_i)P(B_i)$$

4.1.0.8 Bayes' Rule:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

This can be derived from the definition of conditional probability as follows:

$$P(A \cap B) = P(A | B)P(B) = P(B | A)P(A) \Rightarrow P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

Assume law of total probability $\mathcal{H}$, i.e. $H_i \cap H_j = \emptyset$ for all $i \neq j$ and $\bigcup_{i=1}^n H_i = \mathcal{H}$. Also assume that D is the observed data. Then, Bayes' rule can be written as:

$$P(H_i | D) = \frac{P(D | H_i)P(H_i)}{P(D)} = \frac{P(D | H_i)P(H_i)}{\sum_{j=1}^n P(D | H_j)P(H_j)}$$

Here,

- $P(H_i)$ is our **prior belief** in a hypothesis H_i ,
- $P(H_i|D)$ is our **posterior belief** in H_i after observing the data D ,

- $P(D|H_i)$ is the **likelihood** of H_i , and $P(D)$ is the **evidence**.

The concepts above give a general framework for **statistical inference**: we start with a prior belief, collect new observations, and update our belief based on them. This approach has connections to how humans make decisions under uncertainty.

4.2 Random Variables

It is not always convenient to describe sets. We can facilitate this by defining a random variable.

A **random variable** is function $X : \Omega \rightarrow \Lambda$ that maps each elementary event $\omega \in \Omega$ to an element on its **range** $\lambda \in \Lambda$. We can define a probability measure on Λ as follows:

$$P_X(A) := P(\{\omega \in \Omega : X(\omega) \in A\})$$

where $A \subset \Lambda$. This is called the **induced probability measure** of X .

Intuition: measures the outcome as a number.

4.2.0.1 Example of random variable

Consider the experiment of tossing a fair coin three times and denote heads by H and tails by T . The sample space for this experiment is $\Omega = \{HHH, HHT, HTH, HTT, THH, THT, TTH, TTT\}$.

Because the coin is fair

$$P(H) = P(T) = \frac{1}{2}$$

Because the coin tosses are independent events, we have

$$P(HHH) = P(H)P(H)P(H) = \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{8}$$

and likewise for all other outcomes. Let X be the number of heads in a sequence. Then X is a random variable with range $\Lambda = \{0, 1, 2, 3\}$. The induced probability measure of X is:

- 1) $P_X(0) = P(\{TTT\}) = \frac{1}{8}$
- 2) $P_X(1) = P(\{TTH, THT, HTT\}) = \frac{3}{8}$
- 3) $P_X(2) = P(\{HTH, HHT, THH\}) = \frac{3}{8}$
- 4) $P_X(3) = P(\{HHH\}) = \frac{1}{8}$

Let us give another example. Let us make up two random variables from the outcome of the roll of a fair die:

- 1) X is the outcome of the die itself i.e. $\Lambda = \{1, 2, 3, 4, 5, 6\}$.
- 2) Y is whether the outcome of the die is even or odd, i.e. $\Lambda = \{\text{even}, \text{odd}\}$.

Then consider the case of the joint event $P(X \leq 4 \cap Y = \text{even}) = P(\{2, 4\}) = \frac{2}{6}$.

Marginalization

- For the following $P(X = x, Y = y)$ where Y is binary (like even odd)
- Because of this we can drop one of them

$$P(X = x) = \sum_{y \in \{E, O\}} P(X = x \mid Y = y) P(Y = y)$$

- and so

$$P(X = x) = \frac{1}{2} P(X = x \mid Y = E) + \frac{1}{2} P(X = x \mid Y = O)$$

4.3 Expectance and Variance

The **expectation** (a.k.a. **expected value**) of a random variable X is defined as:

$$E[X] := \sum_{x \in \Lambda} x P_X(x)$$

Take the three coin tosses example from 2. Random Variables where the random variable X indicates the number of heads, $E[X] = 1.5$ because $E[X] = 0 \cdot \frac{1}{8} + 1 \cdot \frac{3}{8} + 2 \cdot \frac{3}{8} + 3 \cdot \frac{1}{8} = 1.5$.

The expectation of a random variable is also called its **mean**. The expectation of a random variable is a measure of its **central tendency**. However, it does not tell us anything about the **spread** of the random variable.

This can be measured by the **variance** of a random variable which is defined as:

$$\text{Var}[X] := E[(X - E[X])^2] = E[X^2] - E[X]^2$$

In the above example, $\text{Var}[X] = 0.75$ because $\text{Var}[X] = E[X^2] - E[X]^2 = 0^2 \cdot \frac{1}{8} + 1^2 \cdot \frac{3}{8} + 2^2 \cdot \frac{3}{8} + 3^2 \cdot \frac{1}{8} - 1.5^2 = 0.75$

The square-root of the variance is called the **standard deviation** of a random variable. Standard deviation is commonly denoted by σ . In the above example, $\sigma(x) = \sqrt{0.75} = 0.866$.

4.3.0.1 Moments

Generally speaking, a **moment** of a random variable is defined as:

$$E[(X - E[X])^k]$$

where k is a positive integer. The first moment is the expectation itself. The second moment is the variance. The third moment is called the **skewness** of a random variable. It is a measure of the asymmetry of the distribution of a random variable. The fourth moment is called the **kurtosis** of a random variable. It is a measure of the heaviness of the tails of the distribution of a random variable.

4.3.1 Common rules for Expectance and Variance

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$$

$$\mathbb{E}[c] = c$$

- Discrete

$$\mathbb{E}[g(X)] = \sum_x g(x)P(X = x)$$

- Continuous

$$\mathbb{E}[g(X)] = \int_{-\infty}^{\infty} g(x)f_X(x)dx$$

$$\mathbb{E}[aX] = a\mathbb{E}[X]$$

$$\mathbb{E}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{E}[X_i]$$

$$\text{If } X \perp Y : \quad \mathbb{E}[XY] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$$

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X | Y]]$$

$$\text{Var}[X] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

$$\text{and so } \mathbb{E}[X^2] = \mathbb{E}[X]^2 + \text{Var}[X]$$

$$\text{Var}(aX) = a^2 \text{Var}(X)$$

$$\text{Cov}[X, Y] := E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y]$$

$$\text{Var}(AB) \neq \text{Var}(A) \cdot \text{Var}(B)$$

above only applies if they are **dependent**

$$\text{Var}(AB) = \mathbb{E}[A]^2 \text{Var}(B) + \mathbb{E}[B]^2 \text{Var}(A) + \text{Var}(A)\text{Var}(B)$$

$$\text{Var}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \text{Var}[X_i]$$

4.4 Continuous Random Variables

Now consider the case where $\Lambda = \mathbb{R}$. This means a random variable X maps outcomes $\omega \in \Omega$ to numbers in $\mathbb{R}$.

In this case, we have a **continuous random variable** with the induced probability measure:

$$P_X(A) = P(\{\omega \in \Omega : X(\omega) \in A\})$$

where $A \subset \Lambda = \mathbb{R}$.

This probability measure has the pre-image of A under X on its argument, i.e. $P_{X(A)} = P(X^{-1}(A))$ Where $X^{-1}(A)$ is the pre-image, aka all the inputs that, when put through the function X , give a numbers in A (the ones we care about).

A common situation for using $X : \mathbb{R} \rightarrow \mathbb{R}$ is where events are intervals (a, b) on the range of X . In this case, we can define a probability measure on $\mathbb{R}$ as follows:

$$P_X(a < x < b) := P(\{\omega \in \Omega : X(\omega) \in (a, b)\})$$

where $a, b \in \mathbb{R}$ and $a < b$.

4.4.1 Example

- Bus arrives uniformly at any time in $[0, 60]$ minutes
- $X(w) = w$ (random variable is just the arrival time)
- $A = [0, 15]$ minutes (we want to know the prop. that the bus arrives in between 15 and 45 minutes.)
- Pre-image is then $X^{-1}(A) = [15, 45]$ in Ω
- Probability is therefore:

$$P_X(A) = P(X^{-1}(A)) = P(w \in [15, 45]) = \frac{45-15}{60} = \frac{1}{2}$$

The σ -algebra of $\mathbb{R}$ by the intervals of the form (a, b) is called the **Borel σ -algebra**.

4.4.2 Cumulative distribution function (CDF)

We can also determine probability given that X is $\leq$ some number x :

$$F_X(x) := P(X \leq x) \quad \forall x \in \mathbb{R}$$

Note that this is a non-decreasing function (probabilities goes up as x increases).

4.4.3 Probability density function (PDF)

Often, we are interested in interpreting the behavior of a continuous distribution by investigating how densely probability is packed at each point x . This is the derivative of $F_X(x)$.

$$f_X(x) := \frac{d}{dx} F_X(x) \quad \forall x \in \mathbb{R}$$

Then, by the fundamental theorem of calculus, we have that :

$$F_X(a) = \int_{-\infty}^a f_X(x) dx$$

Hence an event can be described in terms of the probability density function as follows:

$$P(a < X < b) = F_X(b) - F_X(a) = \int_{-\infty}^b f_X(x)dx - \int_{-\infty}^a f_X(x)dx = \int_a^b f_X(x)dx$$

The probability density function of a continuous random variable is non-negative and integrates to 1:

$$\int_{-\infty}^{\infty} f_X(x)dx = 1$$

However, keep in mind that the probability density function can take values greater than 1, since its argument is not a probability but a probability density. The total densities of a **set** of elementary events make up a probability.

Consider that

$$P_X(X = x) = P_X(x < X < x) = \int_x^x f_X(u)du = 0$$

This means that

- There are infinitely many possible events in (a, b)
- Therefore, the probability that an event takes place at exactly (a, a) is 0.

The cumulative distribution function of two jointly distributed random variables X and Y is defined as:

$$F_{X,Y}(x, y) := P(X \leq x, Y \leq y) = \int_{-\infty}^x \int_{-\infty}^y f_{X,Y}(u, v)dvdu$$

where $f_{\{X, Y\}}(x, y)$ is the joint probability density function of X and Y . The probability density function of X can be obtained by taking the partial derivative of $F_{\{X, Y\}}(x, y)$ with respect to x :

$$f_{X,Y}(x, y) := \frac{\partial}{\partial x} F_{X,Y}(x, y) \quad \forall x, y \in \mathbb{R}$$

The expectation of a continuous random variable is defined as:

$$E[X] := \int_{-\infty}^{\infty} x f_X(x)dx$$

The definition of its variance and standard deviation are the same as in the discrete case.

$$\text{Var}[X] := \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$

$$\sigma = \sqrt{\text{Var}(X)}$$

4.5 Common Distributions

- Benoulli
 - Binary output (coinflip, yes/no ect.)
 - $P_X(x) = p^x(1-p)^{1-x}$
 - this way $P_X(1) = p$ and $P_X(0) = 1-p$
 - Then $\mathbb{E}[X] = p$ and $\text{Var}[X] = p(1-p)$
- Beta
 - The Beta distribution elegantly captures both our **belief about the center** and our **confidence in that belief!**

$$f_X(x) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1} (1-x)^{\beta-1}$$

- where $\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)}$ is the beta function $B(\alpha, \beta)^{-1}$
- Note that $\Gamma(x) = (x-1)!$
- $x^{\alpha-1}$: Controls the behavior near $x = 0$
- $(1-x)^{\beta-1}$: Controls the behavior near $x = 1$
- Together they create the characteristic Beta shape
 - **Example:**
 - 1) Prior: Beta(1,1) - “No idea about coin bias”
 - 2) Flip coin 10 times: 7 heads, 3 tails
 - 3) Posterior: Beta(1+7, 1+3) = Beta(8,4)
 - 4) New belief: Coin is probably biased toward heads

- Multi-noulli
 - **Concrete Example: Rolling a die** If XX represents the outcome of rolling a 6-sided die:
 - $P_X(1)$ = “probability of rolling a 1”
 - $P_X(2)$ = “probability of rolling a 2”
 - $P_X(6)$ = “probability of rolling a 6”
 - For a fair die:** $P_X(1) = P_X(2) = \dots = P_X(6) = \frac{1}{6}$ **For a biased die:** Maybe $P_{X(6)} = 0.5$ and others are 0.1 each

- Dirichlet

$$f_X(x) = \frac{\Gamma(\sum_{i=1}^d \alpha_i)}{\prod_{i=1}^d \Gamma(\alpha_i)}$$

- **Example: Website A/B/C Testing**

- 1) **Prior:** Dirichlet(1, 1, 1) - “no preference between variants”
 - 2) **Observe:** Variant A: 20 clicks, B: 15 clicks, C: 10 clicks
 - 3) **Posterior:** Dirichlet(21, 16, 11) - “updated beliefs”
 - 4) repeat
 - 5) At the end use results to predict new observations
- Uniform
 - **The constant $\frac{1}{b-a}$:**
 - This is the **height** of the rectangular probability density
 - **Why this value?** Because the area must equal 1 (total probability)
 - Area = height $\times$ width = $\frac{1}{b-a}(b-a) = 1 \checkmark$
 - $\mathbb{E}[X]$ is the **midpoint** of the interval - so perfectly symmetric
 - $\text{Var}[X]$ Depends on the **width** of the interval
 - Wider interval $\rightarrow$ more spread $\rightarrow$ higher variance
 - **Intuitive Understanding “All outcomes equally likely”** - that’s the essence of uniform distribution.
 -
 - **Examples:**
 - **Fair die:** Each face has probability $\frac{1}{6}$ (discrete uniform)
 - **Random time:** Arrival uniformly distributed between 2:00 PM and 4:00 PM
 - **Random number generator:** Most computers generate uniform random numbers on $[0,1]$
 - Gaussian
 - Mean: μ
 - Standard deviation: σ
 -
$$f_X(x) = \mathcal{N}(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$
 - where
$$P(a < x < b) = \int_a^b f(x) dx$$
 - Here $\frac{1}{\sqrt{2\pi\sigma^2}}$ is the normalization, meaning that
$$\int_{-\infty}^{\infty} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} = \sqrt{2\pi\sigma^2}$$
 - and so
$$\int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} = 1$$
 -
 - Multivariable Gaussian

$$f_X(x) = \mathcal{N}(x|\mu, \Sigma) = \frac{1}{\sqrt{((2\pi)^2)\|\Sigma\|}} \cdot e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

-
- for the parameters

$$\mu = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad \Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

4.6 Estimators And Bias

4.6.0.1 Estimator

A function that estimates a value. Very simple

Ex we can estimate a mean $\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$

4.6.0.2 Estimator Bias:

The **bias** of an estimator f is defined as $B(f) = E[f(S)] - \theta$, where $B(f)$ is Bayes Error

An estimator is said to be **unbiased** if $B(f) = 0$.

However, the sample variance $S^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \hat{X})^2$ is a biased estimator of the variance σ^2 of a random variable X . That is why is it common to use the **unbiased sample variance** $\frac{1}{n-1} \sum_{i=1}^n (X_i - \hat{X})^2$ instead of the sample variance S^2 . The change made in the denominator is called **Bessel's correction**.

Think of it like shooting at a target:

- Unbiased estimator:
 - Your shots are **centered** on the bullseye (true value)
 - Sometimes you're high, sometimes low
 - **On average**, you hit the center
- Biased estimator:
 - Your shots are **systematically off-center**
 - Maybe always shooting a bit to the left
 - **On average**, you miss the target, but... in a predictable direction :O

4.7 Inequalities

Markov's inequality

$$P(X \geq \varepsilon) \leq \frac{\mathbb{E}[X]}{\varepsilon}$$

Therefore $P(X < \varepsilon) > 1 - \frac{\mathbb{E}[X]}{\varepsilon}$

• Key Properties

- **Minimal Requirements:** Only needs $X \geq 0$ - the most general assumption possible.

- ▶ **One-Sided Bound:** Only controls upper tail $P(X \geq \epsilon)$, not deviations around the mean.
- ▶ **No Sample Size Improvement:** The bound doesn't get better with more data.
- ▶ **Often Vacuous:** Can give meaningless bounds like $P \leq 5/3 > 1$.
- ▶ **Why It Matters:**
 - **Starting Point:** Foundation for deriving other inequalities
 - **Worst-Case Analysis:** When you know almost nothing about your random variable
 - **Quick Feasibility Check:** “Is this probability even theoretically possible?”
- ▶ **Intuition:** If you only know the average, you can't say much about extremes. Like knowing the average income in a city tells you little about how many millionaires live there.
- Chebyshev's inequality
 - ▶ With Chebyshev's inequality, we are able to describe the same problem as an expression dependent on n (sample size).
 - ▶ With this, probability of correctness will maybe increase as sample size increases
 - In other words, **the probability that our estimate is “correct” (i.e., close to the true mean) increases as the sample size grows.**
 - This is because of

$$P(|X - \mu| \geq \epsilon) \leq \frac{\sigma^2}{\epsilon^2}$$

by this as $n \rightarrow \infty$, $rhs \rightarrow 0$

- By this to get probability $\leq \delta$, you need $n \leq \frac{\sigma^2}{n\epsilon^2}$ samples

Why This Is Better:

- ▶ **Variance Matters:** If you know values are tightly clustered (small σ^2), deviations are less likely
- ▶ **Sample Size Scaling:** More data $\rightarrow$ better estimates (empirical risk gets closer to true risk)
- ▶ **Practical Bounds:** Often gives more meaningful bounds where Markov fails

Intuition: Knowing both the average AND the spread lets you make much better predictions. Like knowing both average height and that heights cluster tightly around the mean.

-
-
- WLLN (weak law of large numbers)
 - ▶ For i.i.d. samples where, mean μ and $\text{Var}(X) = \sigma^2$
 - ▶ Then sample mean $(\hat{y}_n)$ converges to μ as sample size increases
 - ▶

- Hoeffding's inequality
 - Hoeffding's for intervals not just bounded by $[0,1]$
 -

$$P\left(\frac{1}{n} \sum_{i=1}^n X_i - E\left[\frac{1}{n} \sum_{i=1}^n X_i\right] \geq \epsilon\right) \leq e^{-2n\epsilon^2 / \sum_{i=1}^n (b_i - a_i)^2}$$

•

To get probability $\leq \delta$, you need $n \geq \frac{\ln(\frac{1}{\delta})}{2\epsilon^2}$ samples

The exponential concentration means that as you collect more training data, your confidence that the empirical risk approximates the true risk grows **exponentially fast**, not just linearly. This is why we can make strong theoretical guarantees about generalization in machine learning!

5 Statistical Learning Theory

5.1 Generalization bounds

Remember the basic definitions below.

Definition 5.1 (Generalization error) Given a hypothesis $h \in \mathcal{H}$, a loss function $\ell : Y \times Y \rightarrow [0, 1]$ a data distribution $x, y \sim D$, the generalization error of h is defined as

$$R(h) = P_{x,y \sim D}[\ell(h(x), y)]$$

Definition 5.2 (Empirical error) Given a hypothesis $h \in \mathcal{H}$, a loss function $\ell : Y \times Y \rightarrow [0, 1]$, and a data set $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$. The empirical error of h is defined as:

$$\hat{R}_S(h) = \frac{1}{m} \sum_{i=1}^m \ell(h(x_i), y_i)$$

Using these definitions above and the concentration inequalities we covered in, 7. Inequalities we can derive the following theorem.

Theorem 5.1 (Generalization bound) Let $\mathcal{H}$ be a finite hypothesis set. Then for any $\delta \in (0, 1)$, with probability at least $1 - \delta$ over the choice of $S \sim D^m$, for all $h \in \mathcal{H}$

$$R(h) \leq \hat{R}_S(h) + \sqrt{\frac{\log |\mathcal{H}| + \log \frac{1}{\delta}}{2m}}$$

Proof is in Melih's lecture notes (I do not recommend :))

This inequality is called a generalization bound, because it bounds the generalization error of any hypothesis $h \in \mathcal{H}$ in terms of its empirical error. We can derive a number of interesting consequences from this bound:

- The bound is uniform in the sense that it holds simultaneously for all hypotheses in $\mathcal{H}$.
- The bound is independent of the data distribution D and the loss function ℓ .
- The bound increases logarithmically with the size of the hypothesis set $|\mathcal{H}|$, that is, a richer hypothesis set is more likely to overfit.
- The bound decreases with the size of the training set m , that is, more data is less likely to overfit.
- For a fixed training set size m and two different hypothesis sets $\mathcal{H}_1$ and $\mathcal{H}_2$, the bound prefers the smaller hypothesis set. This is known as **Occam's razor principle**. The principle is introduced by the 14th century theologian William of Ockham. It states that among competing hypotheses, the one with the fewest assumptions should be selected.

This all means that **True error $\leq$ training error + bias** for model complexity. Since training error gets smaller when training, with enough samples m , this penalty can be made as small as desired

- The bound gives a **Probably Approximately Correct (PAC)** performance guarantee. The event that any hypothesis in $\mathcal{H}$ is **approximately correct** in the sense that its generalization error is at most ϵ with probability at least $1 - \delta$.

5.2 PAC Learnability

5.2.0.1 Realizability

A hypothesis space $\mathcal{H}$ is realizable with respect to a loss ℓ and data distribution D if $\exists h^* \in \mathcal{H}$ such that $R(h^*) = 0$. It trivially follows from this definition that $\min_{h \in \mathcal{H}} \hat{R}_S(h) = 0$ with probability 1 for a sample set S collected i.i.d. from D . Hence, under the realizability assumption, all Empirical Risk Minimization (ERM) solutions

$$h_S \in \arg \min_{h \in \mathcal{H}} \hat{R}_S(h)$$

give zero error.

- In other words, the hypothesis class $\mathcal{H}$ is **rich enough** to contain the “true function.” (label function)
- If this is true, it is **possible** to achieve zero training error **and** zero true error.

This is just an assumption that makes PAC learning easier.

5.2.0.2 Representativeness

A training set S is called ϵ -representative if

$$\forall h \in \mathcal{H}, |R(h) - \hat{R}_S(h)| \leq \epsilon$$

This means that all hypothesis in the h -space, their risk must be epsilon similar to the generalization error

Intuition: The dataset is ε -representative if there exists a hypothesis with **minimal** error.

5.2.0.3 Uniform convergence

A hypothesis class $\mathcal{H}$ has the **uniform convergence property** if there exists a function $m_{\mathcal{H}}^{\text{UC}} : (\epsilon, \delta)^2 \rightarrow \mathbb{N}$ such that for every $\epsilon, \delta \in (0, 1)$ and every distribution D , any sampled dataset $S = \{(x_i, y_i) \stackrel{i.i.d.}{\sim} D : i = 1, \dots, m\}$ is ϵ -representative with probability at least $1 - \delta$.

This means that there exists a function that can tell how many data points you would need for a hypothesis, to know that for all distributions. It is ε - representative with a $1 - \delta$ probability certainty

5.2.0.4 Agnostic PAC learnability

A hypothesis class $\mathcal{H}$ is **agnostic PAC learnable** if there exist a function $m_{\mathcal{H}} : (\epsilon, \delta)^2 + h < -A(S)$ and a **learning algorithm** A such that for every $\epsilon, \delta \in (0, 1)$ and every distribution D , running the learning algorithm on dataset $S = \{(x_i, y_i) \stackrel{i.i.d.}{\sim} D : i = 1, \dots, m\}$ with $m \geq m_{\mathcal{H}}(\epsilon, \delta)$ satisfies the following

$$P(\{S \sim D : R(A(S)) \leq \min_{h' \in \mathcal{H}} R(h') + \epsilon\}) \geq 1 - \delta.$$

This means that a hypothesis class $\mathcal{H}$ is **agnostic PAC learnable** if we can guarantee that with enough data, we can find a hypothesis that performs almost as well as the best possible hypothesis in our class.

- 1) “**Agnostic**” = We don’t assume there’s a perfect hypothesis in $\mathcal{H}$ that makes zero errors. We’re realistic about the fact that our hypothesis class might not contain the true underlying function.
 - (if $\min_{\mathcal{H}} R(h) = 0$ and $\mathcal{H}$ is agn. PAC learnable, then $\mathcal{H}$ is PAC learnable)
- 2) “**PAC**” = “Probably Approximately Correct” - we get performance guarantees that are:
 - **Probably**: With high probability (at least $1 - \delta$)
 - **Approximately**: Within ε error of the best possible
 - **Correct**: We can make this guarantee
- 3) **The guarantee**: If we have enough training data ($m \leq \mathcal{H}(\varepsilon, \delta)$), then our learning algorithm A will find a hypothesis that performs within ε of the best possible hypothesis in H .

In practical terms:

- ε = How close we want to be to optimal (smaller = better)
- δ = How confident we want to be (smaller = more confident)

- $m_{\mathcal{H}}(\epsilon, \delta)$ = Minimum number of training examples needed
- The algorithm guarantees: $R(A(S)) \leq \min_{h \in \mathcal{H}} R(h) + \epsilon$ with probability $\geq 1 - \delta$

5.2.0.5 Bayes Error

Bayes error is the best achievable error once unavoidable factors like noise are taken into account. $R^* = \min_{h' \in \mathcal{H}} R(h')$ This means that:

- Even with a perfect model and infinite data, you **cannot do better** than this error.
- The remaining error is due to **irreducible uncertainty**, such as:
 - noise in the data,
 - overlapping class distributions,
 - inherent randomness in the labels.

Then a hypothesis class is **PAC learnable** with respect to a data distribution D if it admits zero Bayes error, i.e., $R^* = 0$.

5.3 Bias-Complexity Dilemma (Trade-off)

5.3.0.1 No Free Lunch

Even though a perfect classifier f exists (with $R(f) = 0$), for any learning algorithm A and some data distribution D . If you randomly sample a training set S from D , there is at least a $\frac{1}{7}$ chance that the algorithm A will output a hypothesis with generalization error at least $\frac{1}{8}$.

No matter how good your algorithm is, there are situations (distributions and training sets) where it will fail to generalize well, even if a perfect solution exists. There is always a non-negligible probability of poor performance.

No algorithm is best at every task

The no free lunch theorem tells us only that we need to include a degree of bias to the learner. However, it does not tell anything about its consequences. Inducing too much bias limits the ability of the learner to explain the training observations. Inducing too little bias leads to overfitting. The goal is to find the right balance between the two. This dilemma is known as the **bias-complexity dilemma**.

5.3.0.2 Bias-complexity dilemma

Bias-complexity dilemma = the trade-off between **how complex your model is** and **how well it generalizes**.

- **Simple model (low complexity)**
 - Cannot capture all the structure in the data → **high bias** / high approximation error
 - Very stable → **low variance** / low estimation error
 - Risk: underfitting
- **Complex model (high complexity)**
 - Can fit the data very well → **low bias** / low approximation error

- ▶ Sensitive to training noise → **high variance** / high estimation error
- ▶ Risk: overfitting

Goal: Pick the complexity where **total generalization error** is minimized.

Let us describe the bias-complexity dilemma in more formal terms. Given an ERM solution $h_S \in \arg \min_{h \in \mathcal{H}} \hat{R}_S(h)$ for a hypothesis space $\mathcal{H}$, we can decompose its generalization error as below

$$\underbrace{R(h_S)}_{\text{Generalization error}} = \underbrace{\min_{h \in \mathcal{H}} R(h)}_{\text{Approximation error}} + \underbrace{\epsilon_{est}}_{\text{Estimation error}}$$

- $R(h_S)$: the true risk (expected error) of the learned hypothesis.
- $\min_{h \in \mathcal{H}} R(h)$: The best possible risk within your chosen hypothesis space. This is also called the **approximation error**.
 - ▶ This comes from the fact that your hypothesis space $\mathcal{H}$ may not be rich enough to perfectly represent the true function.
 - ▶ No matter how much data you have, you can't beat this.
- ϵ_{est} : the **estimation error**, the extra error due to having only a finite training sample S .
 - ▶ This decreases as you get more data, because your empirical risk minimizer h_S gets closer to the true best $h^* \in \mathcal{H}$.
 - ▶ But if $\mathcal{H}$ is large/complex, ϵ_{est} is bigger because you risk overfitting the training data.
 - ▶ We can see ϵ_{est} as a kind of penalty (a **cost of complexity**)

Since for any $\mathcal{H}' \supset \mathcal{H}$, it holds that

$$\min_{h' \in \mathcal{H}'} R(h') \leq \min_{h \in \mathcal{H}} R(h)$$

We can influence the approximation error by choosing different hypothesis from the hypothesis space. Our task is to trade off approximation and estimation error against each other.

Increasing the hypothesis space, thereby making our solution more complex will reduce the approximation error, but will increase the estimation error. Since the realizability assumption (Realizability: 2. PAC Learnability) implies a smaller training error, we will observe overfitting in this scenario. The opposite will be true when we restrict the hypothesis space.

5.3.0.3 Bias-Variance decomposition

The bias-complexity dilemma can also be observed from the bias and variance of the estimated values of a regression output.

Consider a regression problem where observations $(x, y) \sim D$. In this instance we want to estimate the conditional expectation of $f^*(x) = \mathbb{E}(y|x)$

With the training set $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$.

Our **estimator** for $\mathbb{E}[y|x]$ is a hypothesis $h_S \in \mathcal{H}$ that minimizes the mean squared error (MSE). The expected squared error of the prediction made by this estimator over the noisy label y and training sample S is given by:

$$\begin{aligned} E_{S,y|x} \left[\left(y - h_S(x) \right)^2 \right] &= E_{S,y|x} \left[y^2 - 2yh_S(x) + h_S(x)^2 \right] \\ &= E_{\{y|x\}}[y^2] + E_{\{S\}}[h_S(x)^2] - 2E_{\{y|x\}}[y]E_{\{S\}}[h_S(x)] \\ &= E_{\{y|x\}}[y^2] + \text{Var}_{\{y|x\}}[y] + E_{\{S\}}[h_S(x)^2] + \\ &\quad \text{Var}_{\{S\}}[h_S(x)] - 2E_{\{y|x\}}[y]E_{\{S\}}[h_S(x)] \end{aligned}$$

This is the important part, its not crucial to understand everything here

$$= \underbrace{\left(E_{y|x}[y] - E_S[h_S(x)] \right)^2}_{\text{Estimator~Bias}} + \underbrace{\text{Var}_S[h_S(x)]}_{\text{Estimator~Variance}} + \underbrace{\text{Var}_{y|x}[y]}_{\text{Label~noise~variance}}$$

Note that $E_{\{S|x\}}[h_{S(x)}] = E_S[h_{S(x)}]$ and $\text{Var}_{\{S|x\}}[h_{S(x)}] = \text{Var}_S[h_{S(x)}]$ since S is collected independently from x .

5.4 Vapnik - Chervonenkis (VC) Dimension

5.4.0.1 Restriction

Given $S = \{x_1, \dots, x_m\} \subset X$, the following set

$$\mathcal{H}_S = \{(h(x_1), \dots, h(x_m)) : h \in \mathcal{H}\}$$

is called a **restriction** of $\mathcal{H}$ to S . We can do this by

$$|\mathcal{H}| = 2^{|S|}$$

In other words, restriction in the discrete-label case (**don't worry about continuous labels here, trust me**), a **restriction** means evaluating hypotheses **only on a finite dataset**. This produces a **finite set of distinct labelings**, even if the original hypothesis space is infinite.

• Example

Consider restricting simple dataset with 3 points: $S = \{(1, 1), (2, 1), (1, 2)\}$

5.4.0.1.1 Finding the Restriction $\mathcal{H}_S$

For each hypothesis $h \in \mathcal{H}$, we evaluate it on our 3 points to get a label vector $\begin{bmatrix} h(x_1) \\ h(x_2) \\ h(x_3) \end{bmatrix}$.

Different linear classifiers might give:

- Line 1: (+1,+1,+1) - all points positive
- Line 2: (+1,+1,-1) - first two positive, last negative
- Line 3: (+1,-1,+1) - first and last positive, middle negative

- Line 4: (+1,-1,-1) - only first positive
- Line 5: (-1,+1,+1) - only first negative
- Line 6: (-1,+1,-1) - only middle positive
- Line 7: (-1,-1,+1) - only last positive
- Line 8: (-1,-1,-1) - all points negative

Key insight: Even though $\mathcal{H}$ has infinitely many lines, they can only produce **finitely many** distinct labelings on our 3-point dataset.

5.4.0.2 Shattering

$\mathcal{H}$ is said to **shatter** S if $|\mathcal{H}_S| = 2^{|S|}$.

In words, a hypothesis class $\mathcal{H}$ shatters from a dataset S if the restriction of $\mathcal{H}$ to S is the set of all functions from S to $\{0, 1\}$. This means that hypothesis class $\mathcal{H}$ is so expressive that it can achieve **every possible labeling** of the dataset S . use

$$d_{VC} = \max\{m : T_{cal}(\mathcal{H})(m) = 2^m\}$$

Intuition: If your hypothesis class can shatter a dataset, it means your hypothesis class is “complex enough” to memorize any labeling of those points - even completely random noise!

Learning Theory Connection:

- **Good:** Expressive enough to capture complex patterns
- **Bad:** So expressive it can memorize noise $\rightarrow$ overfitting risk
- **Key insight:** There’s a maximum dataset size your hypothesis class can shatter $\rightarrow$ this is the **VC dimension**

5.4.0.3 Growth function

The growth function, $\tau_{\mathcal{H}} : \mathbb{N}^+ \rightarrow \mathbb{N}^+$ of $\mathcal{H}$ is defined as

$$\tau_{\mathcal{H}}(m) := \max_{S \in X^m} |\mathcal{H}_S|$$

The expression above determines the max number of distinct labels in a model that uses a dataset that is on the domain X .

In this case m is the number of data points in the dataset (on X) that the model uses.

When using the growth function to find a shatter-able dataset its to answer the question “**What’s the largest dataset that H can fully control?**”

5.4.0.4 VC dimension

The VC dimension of a hypothesis set $\mathcal{H}$ is the size of the largest dataset that $\mathcal{H}$ can shatter:

$$d_{VC}(\mathcal{H}) = \max\{m : \tau_{\mathcal{H}}(m) = 2^m\}$$

Note that if $d_{VC}(\mathcal{H}) = \infty$ then $\mathcal{H}$ is not PAC learnable.

Theorem 5.4 (The Fundamental Theorem of Statistical Learning). Assume that $d_{VC}(\mathcal{H}) = d < \infty$. Then, there exist $C_1, C_2 \in \mathbb{R}^+$ such that

- $\mathcal{H}$ has the uniform convergence property with sample complexity

$$C_1 \frac{d + \log(1/\delta)}{\epsilon^2} \leq m_{\mathcal{H}}^{\text{UC}}(\epsilon, \delta) \leq C_2 \frac{d + \log(1/\delta)}{\epsilon^2}$$

- $\mathcal{H}$ is agnostic PAC learnable with sample complexity

$$C_1 \frac{d + \log(1/\delta)}{\epsilon^2} \leq m_{\mathcal{H}}(\epsilon, \delta) \leq C_2 \frac{d + \log(1/\delta)}{\epsilon^2}$$

- $\mathcal{H}$ is PAC learnable with sample complexity

$$C_1 \frac{d + \log(1/\delta)}{\epsilon} \leq m_{\mathcal{H}}(\epsilon, \delta) \leq C_2 \frac{d \log(1/\epsilon) + \log(1/\delta)}{\epsilon}$$

In other words, the following statements are equal:

- $\mathcal{H}$ has the uniform convergence property.
- Any ERM rule is a successful agnostic PAC learner for $\mathcal{H}$.
- $\mathcal{H}$ is agnostic PAC learnable.
- $\mathcal{H}$ is PAC learnable.
- Any ERM rule is a successful PAC learner for $\mathcal{H}$.
- $\mathcal{H}$ has a finite VC-dimension.

5.5 Nonuniform Learnability

5.5.0.1 Formal definition

A hypothesis class $\mathcal{H}$ is **nonuniformly learnable** if there exist a function $m_{\mathcal{H}}^{\text{NU}} : (0, 1)^2 \times \mathcal{H} \rightarrow \mathbb{N}$ and a **learning algorithm** A such that for every $\epsilon, \delta \in (0, 1)$ and every distribution D , running A on data set $S = \{(x_i, y_i) \stackrel{i.i.d.}{\sim} D : i = 1, \dots, m\}$ with $m \geq m_{\mathcal{H}}^{\text{NU}}(\epsilon, \delta, h)$ satisfies $P(\{S \sim D : R(A(S)) \leq \min_{h' \in \mathcal{H}} R(h') + \epsilon\}) \geq 1 - \delta$.

Unlike uniform learnability (fx Agnostic PAC), where with enough data, the algorithm **can always find a hypothesis close to the best one in $\mathcal{H}$** . This means that uniform learning prepares for the worst hypothesis, even if the data only needs a simple one.

In **Nonuniformly learnability** different hypotheses might need **different amounts of data**.

We **assume** that **for each** hypothesis $h \in \mathcal{H}$, there exists a **sample size** $m_{\mathcal{H}}^{\text{NU}}(\epsilon, \delta, h)$ that is sufficient to learn it:

$$P(R(A(S))) \leq R(h) + \epsilon \geq 1 - \delta, \quad \text{if } |S| \leq m_{\mathcal{H}}^{\text{NU}}(\epsilon, \delta, h)$$

Note that an agnostic PAC learnable hypothesis class $\mathcal{H}$ is also nonuniform learnable because

$$R(A(S)) \leq \min_{h' \in \mathcal{H}} R(h') + \epsilon$$

results **trivially** in.

$$R(A(S)) \leq R(h) + \epsilon, \forall h \in \mathcal{H}$$

Hence, the set of nonuniform learnable hypothesis classes is larger than the agnostic PAC learnable hypothesis classes. With this relaxation, our motivation is to increase the model capacity while sustaining its learnability.

If in uniform learning there is a very complex (bad) hypothesis in $\mathcal{H}$, then sample complexity explodes (we need huge amounts of data), so we are forced to restrict $\mathcal{H}$ (Restriction: 2. PAC Learnability) in order to keep learnability.

With this new concept that Each hypothesis h has its **own required sample size**, simple hypotheses can be learned with few samples and the complex hypotheses are still allowed — they just “cost more data”

5.5.0.2 Intuition

With limited data, we restrict ourselves to simple hypotheses, then as data grows, we are allowed to consider more complex hypotheses.

6 Bayesian Learning

6.1 Maximum Likelihood Estimation (MLE)

MLE assumes that there is even more uncertainty and randomness than previously covered. MLE is like rewinding the data-generating process and asking which parameters could have most plausibly produced what we observed. (So we can replicate them)

Let

$$S = \{x_1, x_2, \dots, x_m\}$$

At the stage of modeling, we draw a hypothesis about how this data could have been generated. Let our hypothesis be that the data is generated as independent samples a distribution following a parametric density function $p(x \mid \theta)$, where θ is the parameter of the distribution. Then the probability density function of the random variable S representing the occurrence of the dataset is given by

$$p(S \mid \theta) = p(x_1, x_2, \dots, x_m \mid \theta) = \prod_{i=1}^m p(x_i \mid \theta).$$

If we find θ we can predict new data in the same shape as S

The expression above is a density function and is called the **likelihood** of parameters θ given the dataset S .

We can do this by maximizing the likelihood function $p(S | \theta)$.

This uses “log-likelihood”. Where because for $p > 0$, $\log(\prod(P) = \sum(\log(p)))$ This way we can inspect each p . So

$$\theta_{MLE} = \arg \max_{\theta} \sum_{i=1}^m \log p(x_i | \theta).$$

6.1.0.1 Example

Let us give an example. Choose $p(x | \theta) = \mathcal{N}(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$, where $\theta = (\mu, \sigma^2)$.

Then consider that a single term of the log-likelihood function for a data point x_i is given by

$$\log p(x_i | \theta) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x_i - \mu)^2}{2\sigma^2}.$$

We can then maximize the likelihood with respect to μ and σ^2 by setting the gradient of the full likelihood function to zero and solving for μ .

$$\frac{\partial}{\partial \mu} \sum_{i=1}^m \log p(x_i | \theta) = \sum_{i=1}^m \frac{x_i - \mu}{\sigma^2} = 0 \Rightarrow \mu_{MLE} = \frac{1}{m} \sum_{i=1}^m x_i.$$

We can also solve for σ^2 .

$$\frac{\partial}{\partial \sigma^2} \sum_{i=1}^m \log p(x_i | \theta) = -\frac{m}{2\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^m (x_i - \mu)^2 = 0 \Rightarrow \sigma_{MLE}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2.$$

Note that the parameters for a normal distribution is just the mean μ and variance σ^2

Now that we have found the parameters that fits the data μ_{MLE} and σ_{MLE}^2

Given a test input x_* , the learned model predicts the output in the form of a distribution, which is called the **predictive distribution**:

$$y_* | x \sim \mathcal{N}(y_* | w_{MLE}^T \phi(x_*), \sigma_{MLE}^2)$$

This determines how likely an output is given an input. If the likelihood is good, when we can use

$$\hat{y} = w_{MLE}^T \phi(x_*)$$

If we have a safety-critical prediction task, we can also build confidence sets. For instance,

$$\begin{aligned}
P(|y_* - w_{MLE}^T \phi(x_*)| \leq \sigma_{MLE}) &= P(w_{MLE}^T \phi(x_*) - \sigma_{MLE} \leq y_* \leq w_{MLE}^T \phi(x_*) + \sigma_{MLE}) \\
&= \int_{w_{MLE}^T \phi(x_*) - \sigma_{MLE}}^{w_{MLE}^T \phi(x_*) + \sigma_{MLE}} \mathcal{N}(y_* | w_{MLE}^T \phi(x_*), \sigma_{MLE}^2) dy_* \\
&= \int_{w_{MLE}^T \phi(x_*) - \sigma_{MLE}}^{w_{MLE}^T \phi(x_*) + \sigma_{MLE}} \frac{1}{\sqrt{2\pi\sigma_{MLE}^2}} \exp\left(-\frac{(y_* - w_{MLE}^T \phi(x_*))^2}{2\sigma_{MLE}^2}\right) dy_* \\
&= \int_{-1}^1 \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{z^2}{2}\right) dz = 0.68.
\end{aligned}$$

6.2 Bayesian Learning

The MLE approach cannot model the potential dependency of the variance on the input x . That means that because σ^2 is a single number. The problem with having a **single variance** (σ^2) is that it **assumes all inputs have the same uncertainty**, which is often unrealistic. Let me explain carefully.

The only source of uncertainty is on the precise values of the model parameters, which can be mitigated by collecting more data due to the law of large numbers. This ansatz is called **frequentist learning**.

In real-world problems, the modeler is never certain about the model and in many cases they are interested in accounting for this source of uncertainty.

Bayesian learning offers a way to do this. Let us see how it works on the same linear regression problem.

$$w \sim \mathcal{N}(w | 0, \alpha^{-1}I)$$

$$y_i | x_i, w \sim \mathcal{N}(y_i | w^T \phi(x_i), \sigma^2), \quad i = 1, \dots, m.$$

This time we are not assuming that w (the distribution) is a fixed parameter, but a random variable. We start with the prior belief that $w \sim p(w)$. Based on the data, we update our belief about w to $p(w|S)$.

- $p(w)$ is called the **prior distribution**
- and $p(w|S)$ is called the **posterior distribution**.

The goal of **learning** in Bayesian machine learning is to calculate the posterior distribution $p(w|S)$ given the dataset S . This means that we start with a “unbiased” distribution and update it as we gather more information

This is done via the Bayes’ rule:

$$p(w \mid S) = \frac{p(S \mid w)p(w)}{p(S)} = \frac{p(S \mid w)p(w)}{\int p(S \mid w)p(w)dw}.$$

The term in the denominator $p(S) = \int p(S \mid w)p(w)dw$ is called the **evidence** (also called the **marginal likelihood**). This means “Given our model (all possible parameter settings), how likely is it that we would see the data we actually observed?”

- In almost every real-world use case, the evidence is hard to calculate because of complexity. Therefore, we will not be able to calculate the posterior distribution exactly. Instead, we will use approximations. The whole field of Bayesian machine learning is about finding good approximations to the posterior distribution.
- The evidence quantifies the fit of the whole model family to data, as it can be viewed as the average likelihood with respect to the prior distribution. Therefore, it can be used for model selection. For instance, if we have two competing models $p(S, w)$ and $p'(S, w')$, we can choose the one with the higher evidence.

Let us next calculate the posterior distribution for the linear regression problem. We have:

$$\begin{aligned} p(w \mid S) &= \prod_{i=1}^m \mathcal{N}(y_i \mid w^T \phi(x_i), \sigma^2) \mathcal{N}(w \mid 0, \alpha^{-1} I) \\ &= \mathcal{N}(w \mid 0, \alpha^{-1} I) \prod_{i=1}^m \mathcal{N}(y_i \mid w^T \phi(x_i), \sigma^2) \\ &\propto \exp\left(-\frac{1}{2} w^T \alpha I w\right) \prod_{i=1}^m \exp\left(-\frac{1}{2\sigma^2} (y_i - w^T \phi(x_i))^2\right) \\ &\propto \exp\left(-\frac{1}{2} w^T \alpha I w - \frac{1}{2\sigma^2} \sum_{i=1}^m (y_i - w^T \phi(x_i))^2\right) \\ &\propto \exp\left(-\frac{1}{2} w^T \left(\alpha I + \frac{1}{\sigma^2} \sum_{i=1}^m \phi(x_i) \phi(x_i)^T\right) w + \frac{1}{\sigma^2} \sum_{i=1}^m y_i \phi(x_i)^T w\right) \\ &= \mathcal{N}\left(w \mid \left(\alpha I + \frac{1}{\sigma^2} \sum_{i=1}^m \phi(x_i) \phi(x_i)^T\right)^{-1} \frac{1}{\sigma^2} \sum_{i=1}^m y_i \phi(x_i), \left(\alpha I + \frac{1}{\sigma^2} \sum_{i=1}^m \phi(x_i) \phi(x_i)^T\right)^{-1}\right). \end{aligned}$$

A neater notation would be possible considering that the posterior mean calculation involves the inverse of the posterior covariance.

$$p(w \mid S) = \mathcal{N}(w \mid \mu_{post}, \Sigma_{post}),$$

where

$$\Sigma_{post} = \left(\alpha I + \frac{1}{\sigma^2} \sum_{i=1}^m \phi(x_i) \phi(x_i)^T \right)^{-1}$$

$$\mu_{post} = \Sigma_{post} \left(\frac{1}{\sigma^2} \sum_{i=1}^m y_i \phi(x_i) \right).$$

For a new test input x_* , the Bayesian model predicts the output in the form of a distribution:

$$\begin{aligned} p(y_* | S, x_*) &= \int p(y_* | x_*, w) p(w | S) dw \\ &= \int \mathcal{N}(y_* | w^T \phi(x_*), \sigma^2) \mathcal{N}(w | \mu_{post}, \Sigma_{post}) dw \\ &= \mathcal{N}(y_* | \mu_{post}^T \phi(x_*), \sigma^2 + \phi(x_*)^T \Sigma_{post} \phi(x_*)). \end{aligned}$$

The resulting distribution $p(y_* | S, x_*)$ is called the **posterior predictive distribution**. This distribution has some remarkable properties:

- It considers all possible values of w and averages over them proportionally to their posterior probability. This essential and unique property of Bayesian models is called **model averaging**. It is a way of accounting for model uncertainty.
- Its variance depends on the input x_* . Hence, it is able to take into account potential changes in the model confidence in different regions of the input space. This property of a probabilistic model is called **heteroscedasticity**. The Bayesian approach provides this property for free, whereas the frequentist approach requires a separate model for the variance.

In the cases where the predictor is a distribution, there are multiple ways one can use it to make a final prediction. For instance, one can choose the mode of the predictive distribution:

$$\hat{y} = \arg \max_y p(y_* = y | S, x_*).$$

This is called the **Bayes predictor**. One can alternatively choose to take a sample from the predictive distribution:

$$\hat{y} \sim p(y_* | S, x_*).$$

This is called the **Gibbs predictor**. Bayes predictor is known to be the optimal predictor in terms of the expected loss. (Example from lectures of predicted)

6.3 Maximum A-Posteriori Estimation

Much like in Bayesian Linear Regression, with **MAP** we can estimate the **posterior mean** of the weight distribution:

$$w_{\text{MAP}} = \arg \max_w p(w \mid S) = \arg \max_w p(S \mid w) p(w) = \arg \max_w \log p(S \mid w) + \log p(w)$$

The intractable denominator $p(S)$ drops out since it does not depend on w . This makes the training objective tractable.

In Bayesian linear regression, the posterior is Gaussian:

$$p(w \mid S) = \mathcal{N}(w \mid \mu_w, \Sigma_w)$$

where the posterior covariance Σ_w is fully determined by the prior and the data. Once Σ_w is known, the MAP estimate is simply the **posterior mean**:

$$w_{\text{MAP}} = \mu_w = \Sigma_w \left(\frac{1}{\sigma^2} \sum_{i=1}^m y_i \phi(x_i) \right)$$

For a given test input x^* , the MAP estimate predicts the output as:

$$y_* \mid x_* \sim p(y_* \mid w_{\text{MAP}}, x_*)$$

Since the MAP estimate is a **point estimate**, it does not capture the uncertainty in the model parameters. Once you have Σ_w , computing MAP is just **matrix multiplication**.

Because of this, it is not considered a fully Bayesian approach, as it cannot perform model averaging over the posterior.

To do model averaging, you would need the average predictions over the posterior including all its uncertainties.

MAP **ignores this uncertainty**, so it can't capture:

- Variance in predictions due to uncertain parameters
- Risk of overconfident predictions if data is scarce
- True Bayesian model averaging benefits

6.4 Monte Carlo Integration

Consider an integral of the form:

$$\mathbb{E}[f(x)] = I = \int f(x) p(x) dx.$$

for some function $f(x)$ and probability density function $p(x)$. If we can take m samples from $p(x)$, we can approximate the integral by the sample average:

$$I \approx \frac{1}{m} \sum_{i=1}^m f(x_i),$$

where $x_i \sim p(x)$. This is called the **Monte Carlo integration**.

6.5 Generative Models

For a supervised learning problem where the data comes from $x, y \sim D$ for an unknown data distribution D . We can approach the modeling problem in two ways based on what we want to approximate about the data distribution. Consider that the joint distribution factorizes in two ways: $p(x, y) = p(y|x)p(x)$

$$p(x, y) = p(x|y)p(y)$$

We can choose the first factorization and account for $p(x)$ using Monte Carlo integration based on the training samples. Then it suffices to approximate $p(y|x)$. This approach is called **discriminative modeling**. The probabilistic and Bayesian linear regressors we developed above were all discriminative models since they fitted $p(y|w, x)$ to data.

An alternative approach would be to choose the second factorization: $p(x, y) = p(x|y)p(y)$ and aim to approximate both $p(y)$ and $p(x|y)$. Notice that this approach attempts to infer the whole data generating process, where a label is first created and the related input is generated. A real-world example could be a person choosing the digit to draw first and then drawing it. The picture of the resulting drawing because the input of the aimed classifier. This approach is called **generative modeling**.

For instance, we can fit a generative model on a data set $S = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ by choosing a parametric distribution family $p(x | y, \theta)$. For a classification problem, we can approximate $p(y)$ by class frequencies, i.e.

$$P(y = c) = \frac{1}{m} \sum_{i=1}^m \mathbb{1}(y_i = c).$$

and choose the class-conditional distribution to be:

$$p(x | y = c, \theta_c) = p(x | \theta_c),$$

that is, each class shares the same distribution family but has its own parameters. Then we can fit the parameters θ_c on the training samples belonging to class c by maximum likelihood estimation:

$$\theta_c^{MLE} = \arg \max_{\theta_c} \sum_{\{i: y_i = c\}} \log p(x_i | \theta_c).$$

Given a test input x_* , the generative model predicts the output in the form of a distribution:

$$p(y = c | x_*) = \frac{p(x_* | y = c, \theta_c) P(y = c)}{\sum_{c'} p(x_* | y = c', \theta_{c'}) P(y = c')}.$$

6.6 Naive Bayes Classifier

In the generative classification example above, consider the case that the class-conditionals are normal distributions with full covariance matrices:

$$p(x \mid y = c, \theta_c) = \mathcal{N}(x \mid \mu_c, \Sigma_c).$$

Then the training set will be used to fit $d + \frac{d(d+1)}{2}$ parameters per class, where d is the dimensionality of the input. The first term is due to the parameters of the mean and the rest is due to the parameters of the covariance matrix. This is a lot of parameters to fit. We can instead make the simplifying assumption that the class-conditionals of the individual features are independent. Then we have:

$$\begin{aligned} p(x \mid y = c, \theta_c) &= \prod_{j=1}^d p(x_j \mid \theta_{j,c}) \\ &= \prod_{j=1}^d \mathcal{N}(x_j \mid \mu_{j,c}, \sigma_{j,c}^2). \end{aligned}$$

The assumption that the class-conditionals factorize across individual features is called the **naive Bayes assumption**. For the normal distributed class-conditionals used in our example, the resulting model will then have only $2d$ parameters per class. The resulting generative classifier is called a **naive Bayes classifier**. Applied to normal distributed class-conditionals, the naive Bayes classifier follows the same formulas as above with the assumption that the covariance matrix is diagonal.

$$P(x_1 = a, x_2 = b \mid y = c) = \frac{\#(x_1=a) \wedge \#(y=c)}{\#(y=c)} \cdot \frac{\#(x_2=b) \wedge \#(y=c)}{\#(y=c)}$$

$$\begin{aligned} P(\text{underbrace}(y = \\ c^*, \text{"prediction"}) \mid \text{underbrace}((x_1 = a^*, x_2 = b^*), \text{"queryinput"})) &= (P(x_1 = a^* \mid y = \\ c^*) \cdot P(x_2 = b^* \mid y = c^*) \cdot P(y = c^*)) / (\sum_{c^*} P(y = c^*) \cdot P(x_1 = \\ a^* \mid y = c^*) \cdot P(x_2 = b^* \mid y = c^*)) \end{aligned}$$

7 Kernel Methods

7.1 Transforming the input space

Suppose that we have a dataset where we can't easily classify data using a support vector (4. Support Vector Machines (SVM)). We can transform the space using a transformation function, to model the data by using more dimensions. Take the example from the lectures

$$T(x, y) = (x^2, y^2, \sqrt{2}xy)$$

This way we can use a hyperplane of +1 dimensionality to classify the data.

7.2 Kernel Trick

Suppose that $T(x)$ is a mapping to a higher dimension

Note that algorithms like SVM only need **dot products**

That means that with

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Because of the definition of the inner product, we can describe the relation between two points using $k(x_i, x_j)$

Computing $\phi(x)$ explicitly can be **very expensive or infinite-dimensional**

- **Linear Kernel:** $k(x, y) = x^\top y$
- **Polynomial Kernel:** $k(x, y) = (x^\top y + c)^d$
- **Radial Basis Function (RBF) Kernel:** $k(x, y) = \exp(-\frac{\|x-y\|^2}{2\sigma^2})$
- **Sigmoid Kernel:** $k(x, y) = \tanh(\gamma x^\top y + c)$

7.3 Kernel regression

(no notes yet)

7.4 Support Vector Machines (SVM)

Among many possible hyperplanes that may separate the data points of two classes, SVM chooses the one that maximizes the smallest distance between a data point and the hyperplane.

This distance is called the **margin**. The data points that are closest to the hyperplane are called **support vectors**.

Note that we use because of these property, SVM is also called a **maximum margin classifier**. We would normally use k-fold cross validation, and balance the bias-variance tradeoff to ensure a maximum margin.

In this example is 2d, where the support vectors are lines

8 Ensemble Methods

8.1 Bootstrap Aggregation (Bagging)

(no notes yet)

8.2 Boosting

(no notes yet)

8.3 Federated Learning

(no notes yet)